Manual Técnico

Proyecto final – AsistenteIA

Información	Descripción
Universidad	Universidad de San Buenaventura
Programa	Tecnología de Desarrollo de Software – VI semestre
Asignatura	Programación Avanzada y Aplicada
Docente	Sandra Esperanza Sánchez Barragán
Proyecto	AsistenteIA – Asistente académico móvil con IA (.NET MAUI, API REST, SQLite)
Integrantes	Juan Sebastián Garzón Suárez y Nicolás Quiroz

1. Requisitos del entorno

- Visual Studio 2026 (o 2022) con la carga de trabajo **Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET**.
- SDK de .NET 10 (con VS 2022 se puede usar .NET 9 cambiando `net10.0` por `net9.0` en el `.csproj`).
- Windows 10/11 o un emulador Android.
- Paquetes NuGet: `CommunityToolkit.Mvvm` 8.4.0 y `sqlite-net-pcl` 1.9.172.
- Git instalado y una cuenta de GitHub.

2. Compilación y ejecución

1. Clone el repositorio y abra `AsistenteIA.sln` (como solución, no como carpeta).
2. Espere la restauración de paquetes NuGet.
3. Seleccione **Windows Machine** (o un emulador Android) y pulse ▶.
4. En la pestaña Configuración guarde su clave de https://gen.pollinations.ai (la clave **no** está en el repositorio).
5. Para generar el ejecutable use **Compilar → Publicar** (Windows) o **Archivar** (Android).

3. Estructura del proyecto

```
AsistenteIA/
  Models/       ChatModels.cs, ConversationRecord.cs, PromptTemplate.cs
  Services/     AiService.cs, ApiKeyService.cs, DatabaseService.cs, AppSettings.cs
  ViewModels/   ChatViewModel, HistoryViewModel, DetailViewModel, SettingsViewModel
  Views/        HomePage, ChatPage, HistoryPage, DetailPage, SettingsPage
  Platforms/    Android y Windows (código de arranque por plataforma)
  Resources/    Styles (Colors.xaml, Styles.xaml), AppIcon, Splash
  AppShell.xaml, App.xaml, MauiProgram.cs, AsistenteIA.csproj
```

4. Componentes clave

- **AiService.AskAsync:** valida el prompt, comprueba la conectividad, lee la clave, arma `ChatRequest`, envía el POST, llama a `EnsureSuccessStatusCode` y devuelve `choices[0].message.content`.
- **ChatViewModel:** controla `IsBusy` y el `CancellationTokenSource`; traduce las excepciones a mensajes (401/403, 429, sin red, cancelación).
- **DatabaseService:** inicializa la conexión de forma perezosa y crea la tabla al primer uso.
- **ApiKeyService:** guarda, lee y elimina la clave con `SecureStorage`.
- **Navegación:** pestañas en `AppShell.xaml`; `DetailPage` se registra con `Routing.RegisterRoute` y recibe `id` por `QueryProperty`.

5. Cambiar de proveedor o de modelo

Cambie `BaseAddress` en `MauiProgram.cs` y elija el modelo en Configuración (o modifique `AppSettings.DefaultModel`). Si el proveedor usa otro formato de autenticación o de respuesta, solo debe ajustarse `AiService` y los modelos de `ChatModels.cs`.

6. Seguridad

- Ninguna clave en el código ni en el repositorio; `.gitignore` excluye `bin/`, `obj/`, `.vs/` y las bases de datos locales.
- No registrar la clave en consola ni incluirla en capturas.
- Si una clave se sube por error al repositorio, revóquela en el proveedor y genere otra; borrar el archivo no basta porque queda en el historial de Git.

7. Repositorio Git

Rama / práctica	Uso
`main`	Versión estable y entregable.
`develop`	Integración del trabajo de cada sprint.
`feature/*`	Una rama por funcionalidad e integrante.
Pull Request	Revisión del otro integrante antes de integrar.

Mensajes de commit sugeridos: `inicio-proyecto`, `feat: modelos JSON`, `feat: ApiKeyService con SecureStorage`, `feat: AiService con POST`, `feat: ChatViewModel con cancelación`, `feat: Shell y cinco pantallas`, `feat: historial SQLite CRUD`, `test: casos P01-P10`, `docs: manuales y matriz de riesgos`.

8. Lista de verificación antes de entregar

- La solución compila y se ejecuta sin errores.
- No existen carpetas `bin/` ni `obj/` en el repositorio.
- No hay claves en el código ni en el historial de Git.
- Ambos integrantes tienen commits.
- Las diez pruebas están ejecutadas y registradas.